Simulation

2026-05-09

Load all the necessary R packages and helper functions

```
library(ggplot2)
library(tidyr)
library(dplyr)
library(purrr)
library(ggsci)
library(tibble)
library(future)
library(future.apply)
library(data.table)
library(bench)
library(MixtureInf2.0)

# Source a function that computes the MLE for a beta mixture
source("functions/mle.beta.R")

# Source a function that computes the PMLE for a beta mixture
# with random initialization.
source("functions/mle.beta.R")

# Source a function that computes the 1-Wasserstein distance
# between two beta mixtures
source("functions/wasserstein1d.beta.mixture.R")

# Source a function that computes the Jensen-Shannon divergence
# of the mixing proportions between two beta mixtures.
source("functions/js_divergence.R")
```

Simulation for comparing different initilization methods, estimators and the computational cost

For the data-generation process, we consider the following distributions:

$$G^* = \{\pi = (0.3, 0.3, 0.4),\ \alpha = (2, 6, 20),\ \beta = (8, 3, 20)\}.$$

```
# Simulation code for the three-component beta mixture example
# in the paper (n = 1000). Depending on the sample size,
# this simulation may take 1-2 hours for n = 1000. The setup
# is the same as in the initialization method comparison; the
# only change is replacing the mle/sam/sar with pmle.beta.rand.
```

```r
set.seed(520)
m0 <- 3
nreps <- 1000
n <- 1000

pi_true <- c(.30, .30, .40)
alpha_true <- c(2, 6, 20)
beta_true <- c(8, 3, 20)

theta_true <- c(pi_true, alpha_true, beta_true)

# canonical_from_list: Convert a fitted beta mixture object to a canonical
# parameter vector (pi, alpha, beta) with components ordered by alpha.
#
# Arguments
#   fit : list-like object with fields
#           - mix_prop : numeric length m0; nonnegative mixing weights
#           - alpha : numeric length m0; beta shape1 parameters (>0)
#           - beta : numeric length m0; beta shape2 parameters (>0)
#   m0 : integer; number of mixture components.
#
# Returns
#   numeric vector length 3*m0: c(pi_sorted, alpha_sorted, beta_sorted), where
#   pi_sorted sums to 1 and sorting is by increasing alpha.
canonical_from_list <- function(fit, m0 = 3) {
  mix <- fit$mix_prop / sum(fit$mix_prop)
  a <- fit$alpha
  b <- fit$beta
  idx <- order(a)
  c(mix[idx], a[idx], b[idx])
}

ests <- list(
  pmle = matrix(NA, nreps, 3 * m0),
  mle = matrix(NA, nreps, 3 * m0),
  sam = matrix(NA, nreps, 3 * m0),
  sar = matrix(NA, nreps, 3 * m0)
)

wass <- matrix(NA_real_, nreps, length(ests))
colnames(wass) <- names(ests)

for (r in seq_len(nreps)) {
  x <- rmix.beta(n, pi_true, alpha_true, beta_true)
  ests$pmle[r, ] <- canonical_from_list(pmle.beta(
    x, m0, an = n^.5))
  ests$mle[r, ] <- canonical_from_list(mle.beta(x, m0))
  ests$sam[r, ] <- canonical_from_list(sam.beta.mix(
    x, m0, epsilon=1,an = n^.5))
  ests$sar[r, ]   <- canonical_from_list(sar.beta.mix(
    x, m0, epsilon=1,an = n^.5))
}
```

```r
for(k in names(ests))
  wass[,k] <- apply(ests[[k]],1,wasserstein1d.beta.mixture,
                    F_par=theta_true)

wass_df <- as_tibble(wass) %>% mutate(rep=row_number()) %>%
  pivot_longer(-rep,names_to="estimator",values_to="wass")

wass_df_1000 <- mutate(wass_df, n = "n = 1000") %>%
  mutate(estimator = toupper(estimator))

js_df_1000 <- imap_dfr(ests, function(mat, est) {
  tibble(
    estimator = toupper(est),
    replicate = seq_len(nrow(mat)),
    js = map_dbl(seq_len(nrow(mat)), function(i)
      js_divergence(mat[i, 1:3], pi_true))
  )
}) %>% mutate(n = "n = 1000")

rmsle_list_1000 <- lapply(ests, function(mat) {
  true_mat <- matrix(theta_true, nrow = nreps, ncol = 9, byrow = TRUE)
  err2 <- (log(mat) - log(true_mat))^2
  rmsle_all <- sqrt(colSums(err2) / nreps)
  rmsle_all[4:9]
})

# To obtain results for n = 100, rerun the simulation above after
# setting `n = 100`, then bind the resulting output
# with the `n = 1000` results.

est_levels <- c("MLE" ,"PMLE", "SAM", "SAR")
pal <- pal_lancet()(length(est_levels))
names(pal) <- est_levels

wass_all <- bind_rows(wass_df_100, wass_df_1000) |>
  mutate(estimator = factor(estimator, levels = est_levels))

# Fig 4: Wasserstein distance comparison between
# PMLE, MLE, SAM and SAR based on data generated with
# a sample size of 100 and 1000
wass_comp <- ggplot(wass_all, aes(estimator, wass, colour = estimator)) +
  geom_boxplot(outliers = FALSE) +
  scale_colour_manual(values = pal) +
  ylab("Wasserstein distance") + xlab("") +
  facet_wrap(~n, scales = "free_y") +
  labs(colour = "Estimators") +
  theme_bw() +
  theme(legend.position = "right", axis.text.x = element_blank())

ggsave("wass_comp.pdf", wass_comp, width = 7, height = 3)

df <- data.frame(
  Estimator = rep(c("MLE", "PMLE", "SAM", "SAR"), each = 6),
```

```
  Parameter = rep(c("alpha1", "alpha2", "alpha3",
                    "beta1", "beta2", "beta3"), times = 4),
  RMSLE_100 = c(rmsle_list_100$mle,
                rmsle_list_100$pmle,
                rmsle_list_100$sam,
                rmsle_list_100$sar),
  RMSLE_1000 = c(rmsle_list_1000$mle,
                 rmsle_list_1000$pmle,
                 rmsle_list_1000$sam,
                 rmsle_list_1000$sar)
)

df_long <- pivot_longer(df, cols = starts_with("RMSLE"),
                        names_to = "SampleSize", values_to = "RMSLE")
df_long$SampleSize <- factor(df_long$SampleSize,
                             levels = c("RMSLE_100", "RMSLE_1000"),
                             labels = c("n = 100", "n = 1000"))

# Fig 5: RMSLE of the estimated shape parameters for
# different estimators with sample sizes of n = 100 and n = 1000.
RMSLE_comp <- ggplot(df_long, aes(x = Parameter, y = RMSLE,
                                  fill = Estimator)) +
  geom_bar(stat = "identity", position = position_dodge()) +
  facet_wrap(~ SampleSize, scales = "free_y") +
  scale_fill_manual(values = pal) +
  theme_bw() +
  theme(axis.title.x = element_blank()) +
  labs(y = "RMSLE")

ggsave("rmsle_comp.pdf", RMSLE_comp, width = 8, height = 4)

js_all <- bind_rows(js_df_100, js_df_1000) %>%
  mutate(estimator = factor(estimator, levels = est_levels))

# Fig 6: JS divergence comparison for the mixing proportion
# between PMLE, MLE, SAM and SAR based on data generated
# with a sample size of 100 and 1000
js_comp <- ggplot(js_all, aes(estimator, js, colour = estimator)) +
  geom_boxplot(outliers = FALSE) +
  scale_colour_manual(values = pal) +
  ylab("Jensen-Shannon divergence") + xlab("") +
  facet_wrap(~n, scales = "free_y") +
  labs(colour = "Estimators") +
  theme_bw() +
  theme(legend.position = "right", axis.text.x = element_blank())

ggsave("js_weight_comp.pdf", js_comp, width = 7, height = 3)

# Simulation code to compare computational time across
# PMLE and moment-based estimators. Runtime depends
# on the number of CPU cores used; in general,
# this simulation can take a long time.
```

```r
sample_sizes <- c(100, 500, 1000, 2000)
component_counts <- 2:5
n_reps <- 1000
epsilon <- 1

n_workers <- as.integer(Sys.getenv("SLURM_CPUS_ON_NODE",
                                   unset = future::availableCores()))

plan(multicore, workers = n_workers)


grid <- CJ(n = sample_sizes,
           m = component_counts,
           rep = seq_len(n_reps))

# safe_time: Evaluate an expression and return elapsed (real) time in seconds.
#
# Arguments
#   expr : expression to evaluate.
#
# Returns
#   numeric scalar; elapsed real time in seconds.
safe_time <- function(expr) {
  tryCatch(
    as.numeric(bench::bench_time(expr)[["real"]], unit = "s"),
    error = function(e) NA_real_
  )
}
safe <- function(fun, ...) safe_time(fun(...))

results <- rbindlist(
  future_lapply(
    seq_len(nrow(grid)),
    function(i) {
      n <- grid$n[i]
      m0 <- grid$m[i]
      an <- sqrt(n)
      x <- rbeta(n,2,2)

      t <- c(
        safe(pmle.beta , x, m0 = m0, an = an, epsilon = epsilon),
        safe(sar.beta.mix, x, m0 = m0, an = an, epsilon = epsilon),
        safe(sam.beta.mix, x, m0 = m0, an = an, epsilon = epsilon)
      )

      data.table(
        n = rep(n, 3L),
        m = rep(m0, 3L),
        rep = rep(grid$rep[i], 3L),
        expr = c("pmle", "sar", "sam"),
        time_s = t
      )
    },
```

```
    future.seed = TRUE,
    future.scheduling = 1
  )
)

results[, expr := factor(expr, levels = c("pmle", "sar", "sam"))]

plot_tbl <- results[!is.na(time_s), .(
  median_s = median(time_s, na.rm = TRUE),
  p25_s = quantile(time_s, .25, na.rm = TRUE),
  p75_s = quantile(time_s, .75, na.rm = TRUE)
), by = .(expr, n, m)]

plot_tbl <- plot_tbl |>
  mutate(expr = toupper(expr)) |>
  mutate(expr = factor(expr, levels = est_levels))

# Fig 7: Computation time for each estimator across
# combinations of sample size and model order.
comp_cost <- ggplot(plot_tbl, aes(x = n, colour = expr, fill = expr)) +
  geom_ribbon(aes(ymin = p25_s, ymax = p75_s), alpha = .12, colour = NA) +
  geom_line(aes(y = median_s)) +
  geom_point(aes(y = median_s), size = 2) +
  facet_wrap(~m, scales = "free", labeller =
               as_labeller(\(x) paste0(x, "-component"))) +
  scale_x_log10(breaks = sample_sizes) +
  scale_colour_manual(values = pal) +
  scale_fill_manual(values = pal) +
  labs(x = "Sample size", y = "Time", colour = "Estimators") +
  guides(fill = "none") +
  theme_bw() +
  theme(legend.position = "right")

ggsave("comp_cost.pdf", comp_cost, width = 7, height = 4)
```

Simulation code for the BMR section

For the data-generation process, we consider the following distributions:

$$G_1 = \{\alpha = 2,\ \beta = 2\},$$

$$G_2 = \{\pi = (0.3, 0.3, 0.3, 0.1),\ \alpha = (3, 4, 3.5, 7),\ \beta = (5, 6, 5.5, 2)\}.$$

```
# Simulation code to compare computational time between
# the traditional order selection method and BMR.
# Runtime depends on the number of CPU cores used; in general,
# this simulation can take a long time.

set.seed(520)
sample_sizes <- c(100, 500, 1000, 2000)
n_reps <- 1000
```

```r
epsilon <- 1
orders <- 3:1
n_workers <- as.integer(Sys.getenv("SLURM_CPUS_ON_NODE",
                                   unset = future::availableCores()))
plan(multicore, workers = n_workers)

grid <- CJ(n = sample_sizes,
           rep = seq_len(n_reps))

safe_time <- function(expr) {
  tryCatch(
    as.numeric(bench::bench_time(expr)[["real"]], unit = "s"),
    error = function(e) NA_real_
  )
}
safe <- function(fun, ...) safe_time(fun(...))

results <- rbindlist(
  future_lapply(
    seq_len(nrow(grid)),
    function(i) {
      n  <- grid$n[i]
      an <- sqrt(n)
      # x <- rbeta(n, 2, 2)  # Use this for G1.
      x  <- rmix.beta(n,c(0.3, 0.3, 0.3, 0.1),
                      c(3, 4, 3.5, 7),c(5, 6, 5.5, 2)) # Use this for G2.

      fit4 <- try(pmle.beta(x, m0 = 4, an = an,
                            epsilon = epsilon), silent = TRUE)
      if (inherits(fit4, "try-error")) {
        orig_weights <- orig_alphas <- orig_betas <- NULL
      } else {
        orig_weights <- fit4$mix_prop
        orig_alphas  <- fit4$alpha
        orig_betas   <- fit4$beta
      }

      t_pmle_tot <- sum(vapply(orders, function(m)
        safe(pmle.beta, x, m0 = m, an = an,
             epsilon = epsilon), numeric(1)))

      t_bmr_tot <- if (is.null(orig_weights))
        NA_real_
      else
        sum(vapply(orders, function(M)
          safe(BMR.CTD, orig_weights, orig_alphas,
               orig_betas, M = M), numeric(1)))

      data.table(
        n = rep(n, 2L),
        rep = rep(grid$rep[i], 2L),
        method = c("PMLE", "BMR"),
        time_s = c(t_pmle_tot, t_bmr_tot)
```

```
      )
    },
    future.seed = TRUE,
    future.scheduling = 1
  )
)

results[, method := factor(method, levels = c("PMLE", "BMR"))]

plot_tbl <- results[!is.na(time_s), .(
  median_s = median(time_s, na.rm = TRUE),
  p25_s = quantile(time_s, .25, na.rm = TRUE),
  p75_s = quantile(time_s, .75, na.rm = TRUE)
), by = .(method, n)]

comp_cost_g2 <- plot_tbl

# To obtain results for G1, change `x` to `rbeta(n, 2, 2)`
# above and rerun the simulation.

df <- bind_rows(
  comp_cost_g1 %>% mutate(group = "G1"),
  comp_cost_g2 %>% mutate(group = "G2")
) %>%
  mutate(
    method = factor(method, levels = c("PMLE", "BMR")),
    n = as.numeric(n),
    median_s = as.numeric(median_s),
    p25_s = as.numeric(p25_s),
    p75_s = as.numeric(p75_s)
  )

# Computational time comparison plot: original method vs. BMR.
g1g2_comp <- ggplot(df, aes(x = n, y = median_s,
                            color = method, group = method)) +
  geom_ribbon(aes(ymin = p25_s, ymax = p75_s, fill = method),
              alpha = 0.12, color = NA) +
  geom_line(aes(y = median_s)) +
  geom_point(aes(y = median_s), size = 2) +
  scale_x_log10(breaks = sample_sizes) +
  facet_wrap(~ group, ncol = 2) +
  labs(
    x = "Sample size",
    y = "Time (s)",
    color = "Methods",
    fill  = "Methods"
  ) +
  theme_bw(base_size = 12) +
  theme(
    panel.grid.minor = element_blank(),
    legend.position = "right"
  )
```

```r
ggsave("comp_cost_bmr.pdf", g1g2_comp, width = 8, height = 4)
```

Application

The datasets included here consist of benign and tumour samples from 4 patients, obtained from the NCBI Gene Expression Omnibus (GEO) repository (GSE119260).

```r
# Code for the application section. We used
# patient 1 here as an illustrative example.
# This simulation takes around 45 minutes to run
# due to the large sample size.

# Basic data cleaning
x_benign <- as.numeric(benign1$VALUE)
x_benign <- x_benign[!is.na(x_benign)]
x_tumour <- as.numeric(tumour1$VALUE)
x_tumour <- x_tumour[!is.na(x_tumour)]

component_labels <- c("Low", "Medium", "High")

make_density_dfs <- function(sample_name, x, plist, alist, blist) {
  x_grid <- seq(0, 1, length.out = 1000)

  df_density <- map_dfr(1:3, \(k) {
    tibble(
      sample = sample_name,
      x = x_grid,
      component = component_labels[k],
      weighted_density = plist[k] * dbeta(x_grid, alist[k], blist[k])
    )
  })

  df_mix <- tibble(
    sample = sample_name,
    x = x_grid,
    density = rowSums(map_dfc(1:3, \(k) plist[k]
                              * dbeta(x_grid, alist[k], blist[k])))
  )

  df_values <- tibble(sample = sample_name, x = x)

  list(values = df_values, density = df_density, mix = df_mix)
}

pmle_benign <- pmle.beta(x_benign,3)
pmle_tumour <- pmle.beta(x_tumour,3)

benign <- make_density_dfs(
  "Benign",  x_benign,
  plist = pmle_benign$mix_prop,
  alist = pmle_benign$alpha,
  blist = pmle_benign$beta
```

```
)

tumour <- make_density_dfs(
  "Tumour",  x_tumour,
  plist = pmle_tumour$mix_prop,
  alist = pmle_tumour$alpha,
  blist = pmle_tumour$beta
)

df_vals <- bind_rows(benign$values, tumour$values)
df_density <- bind_rows(benign$density, tumour$density)
df_mix <- bind_rows(benign$mix, tumour$mix)

# Fig 10: Density and histogram plot of DNA methylation levels
# from benign and tumour samples of Patient 1
patient1_den <- ggplot(df_vals, aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins = 200, fill = "gray80", color = "gray60") +
  geom_line(data = df_mix,
            aes(y = density),
            colour = "black", linewidth = 1.2) +
  geom_line(data = df_density,
            aes(y = weighted_density, colour = component),
            linewidth = 1, alpha = 0.8) +
  scale_y_continuous(expand = expansion(c(0, 0.05))) +
  labs(x = "Beta values",
       y = "Density",
       colour = "Inferred methylation category") +
  facet_wrap(~sample, ncol = 2) +
  theme_bw() +
  theme(legend.position = "bottom")

ggsave("patient1.pdf", patient1_den, width = 9, height=4)

# thres: Determination of threshold values for classifying
# DNA methylation levels as low, medium, or high.
#
# Arguments
#   x : numeric vector length n; observations in (0,1).
#   plist : numeric vector length m0; nonnegative mixing weights for components.
#   alist : numeric vector length m0; beta shape1 parameters (>0).
#   blist : numeric vector length m0; beta shape2 parameters (>0).
#
# Returns
#   Numeric vector length 2 with elements:
#     threshold1 : low/medium cutpoint (0 < threshold1 < threshold2)
#     threshold2 : medium/high cutpoint (threshold1 < threshold2 < 1)
thres <- function(x, plist, alist, blist) {
  m1 <- alist[1] / (alist[1] + blist[1])
  m2 <- alist[2] / (alist[2] + blist[2])
  m3 <- alist[3] / (alist[3] + blist[3])

  f12 <- function(x) plist[1] * dbeta(x, alist[1], blist[1]) -
```

```
    plist[2] * dbeta(x, alist[2], blist[2])
  f23 <- function(x) plist[2] * dbeta(x, alist[2], blist[2]) -
    plist[3] * dbeta(x, alist[3], blist[3])

  threshold1 <- uniroot(f12, lower = m1, upper = m2)$root
  threshold2 <- uniroot(f23, lower = m2, upper = m3)$root

  return(c(threshold1,threshold2))
}

rousignif(thres(x_benign,
                plist = pmle_benign$mix_prop,
                alist = pmle_benign$alpha,
                blist = pmle_benign$beta))

rousignif(thres(x_tumour,
                plist = pmle_tumour$mix_prop,
                alist = pmle_tumour$alpha,
                blist = pmle_tumour$beta))
```